

# Coexistence, Not Coupling: A Standalone Specification of Pattern C (Adjacent Component as Separate Concern) in the Coordination Knowledge Substrate Pattern

*A derivation note from the Coordination Knowledge Substrate (CKS) pattern.*

**Author:** Wenxin Li (Independent Researcher)

**ORCID:** 0009-0004-8065-3235

**Date:** May 5, 2026

**License:** Creative Commons Attribution 4.0 International (CC BY 4.0)

---

## Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate, in operational form, the architectural content of Pattern C — the position in which an adjacent AI component coexists with a CKS substrate as a separate concern — so that downstream work can adopt, implement, or argue against the pattern without ambiguity, and so that the most common Pattern C failure mode (silent scope creep) becomes architecturally identifiable.

## Abstract

The CKS hybrid-systems-composition specification supports exactly three positions an adjacent AI component can occupy relative to a substrate: as input to a cell (Pattern A), as a derived view of substrate content (Pattern B), or as a separate concern (Pattern C). This note formalizes Pattern C as a standalone architectural commitment, independent of the other two. Pattern C names the position in which an adjacent AI component handles a distinct concern — reference-document retrieval, semantic search over reference material, domain-specialized inference, language tasks the substrate does not represent — that does not affect coordination state, with the two systems coexisting without architectural coupling. The note specifies the four operational components of the pattern (the component handles non-coordination work only; holds no coordination state; exercises no governance authority; the boundary between its domain and the substrate's domain is inspectable), distinguishes Pattern C from adjacent variations commonly conflated with it, names the failure modes that violate Pattern C through scope creep, and provides an operational test for whether a separate-concern boundary satisfies the pattern at any time during a deployment's existence.

## 1. Why Pattern C needs to be formalized as standalone

The parent foundational note A1.16 commits to three composition patterns; the integrating-frame note A2.91 named Pattern C alongside Patterns A and B; A2.92 formalized Pattern A and A2.93 formalized Pattern B. This note completes the pattern-specialization triple by formalizing Pattern C as having independent architectural content, with particular weight on the dynamic that distinguishes Pattern C violations from coherent Pattern C uses: silent scope creep over deployment lifecycle.

The motivating cases are operationally common: a regulated coordination workflow with a CKS substrate at its center sits beside a RAG retrieval system answering reference-document queries unrelated to the coordination work, a fine-tuned LLM used for tasks the substrate does not represent, a vector database supporting semantic search over reference materials, or a domain-specialized inference system handling non-coordination tasks. Pattern C is not the rare composition; it is the typical one in 2024–2026 hybrid AI deployments. The architectural question is not whether such coexistence happens but whether it remains coexistence rather than drifting into coupling.

The standalone treatment matters because the inspectable-boundary commitment per Pattern C preserves human governance per A1.01 across separate-concern compositions; without inspectability, an adjacent component is in practice outside human governance regardless of how innocuous its current behavior appears, which violates the source paper's first commitment (§3.1, §3.3) and depends specifically on the inspect right per A2.01. Pattern C and Pattern A are also operationally distinct positions; components must occupy one or the other clearly, with components that drift into coordination scope re-evaluated under Pattern A per A2.92.

## 2. Pattern C, defined precisely

In the CKS pattern, an adjacent AI component is in **Pattern C** if and only if it handles a distinct concern that does not affect coordination state and the two systems coexist without architectural coupling. The pattern resolves into four operational components, each architecturally load-bearing.

**(a) The component handles non-coordination work only.** The adjacent component's scope is restricted to work that is operationally distinct from coordination — reference-document retrieval, semantic search over reference materials, domain-specialized inference, language tasks the substrate does not represent. The component does not answer coordination questions, produce coordination decisions, or modify coordination state. Operational distinctness is required; nominal labeling is not sufficient.

**(b) The component does not hold coordination state.** The categories of state the substrate is the source of truth for per A2.42–A2.48 must not migrate to the component. The illustrative test from §5 of the foundational note is sharp: a reference-document retrieval system that answers "find me passages about topic X" without recording what was decided about topic X is Pattern C; the same system used to answer "what did we decide about topic X" has crossed into coordination scope and must be evaluated under Pattern A.

**(c) The component does not exercise governance authority.** If the component is involved in coordination decisions, its involvement must be modeled as Pattern A, not as autonomous decision-making. A fine-tuned model used to draft text the substrate then carries is acting as input to a cell (Pattern A); a fine-tuned model writing decisions into substrate fields under no orchestration rule is exercising governance authority the architecture does not grant.

**(d) The boundary between the component's domain and the substrate's domain is inspectable.** A human exercising the inspect right per A2.01 (referenced at §3.1, §3.3) must be able to determine which decisions the substrate carries and which the adjacent component handles, without ambiguity. The inspectability is architectural; it must be operationally enforced, not merely claimed in documentation. If the boundary is not inspectable, the adjacent component is in practice outside human governance regardless of whether the component happens to behave well.

Pattern C is operationally distinct from Patterns A and B because the adjacent component does not interact with substrate or cells in the architectural sense — Pattern A involves cells consulting adjacent components during execution, Pattern B involves derived views serving as access paths, Pattern C involves no architectural interaction. The commitment is to a non-interaction boundary that is inspectable, ensuring nothing crosses into coordination scope without being recognized.

By definition, Pattern C preserves the foundational CKS commitments without interference at the substrate boundary: AI-as-substrate-mediator per A1.04 is not affected because no LLM-substrate interaction occurs through the Pattern C component, and path retraceability per A1.07 is not affected because no coordination-state writes pass through the component. The temporal property per A2.07 governs the inspectability requirement, which must hold at any time during the deployment's existence, not only at scheduled review windows.

### 3. What Pattern C is NOT

The definition above is precise about what Pattern C *is*. It is equally important to state what it is not, because each of the following is a real configuration in some deployment, and conflating any of them with Pattern C produces a misreading of the position.

**Not a claim about deployment composition.** Pattern C does not require all hybrid compositions to use it; Patterns A and B are also legitimate. It does not require every CKS deployment to include separate-concern adjacent components. It does not foreclose operational coexistence — adjacent components and CKS may share infrastructure, hosting, and identity systems; the commitment is to architectural non-coupling, not operational separation. It does not specify implementation patterns; the architectural commitment is to the four components, not to specific enforcement mechanisms.

**Not Pattern C with implicit coordination scope.** Compositions in which an adjacent component nominally handles non-coordination work but operationally answers coordination questions through general-purpose capability fail component (a). A "document search system" that, when prompted, answers questions about what was decided is no longer in Pattern C regardless of how its product description reads.

**Not vendor-bundled adjacent components without inspectability.** Compositions in which vendor-bundled "AI assistants" or "AI agents" run alongside CKS without explicit scope clarity fail component (d). Pattern C requires humans to determine the component's scope through the inspect right per A2.01; vendor-bundled components whose behavior is opaque or whose scope is set by the vendor rather than the deployment do not satisfy this regardless of how well they appear to behave.

**Not "general-purpose AI" running alongside CKS without scope clarity.** Compositions that include general-purpose AI capabilities that may be invoked for coordination decisions through operational drift fail components (a) through (c). Pattern C requires explicit scope restriction; general-purpose capability without scope restriction does not satisfy the pattern.

**Not adjacent components handling administrative-but-coordination-adjacent tasks.** Compositions in which adjacent components handle work that is operationally administrative but architecturally adjacent to coordination — for example, an access-management AI that decides who can write to substrate fields — fail component (c). Administrative tasks that affect coordination authority are coordination work, and their components belong in Pattern A scope.

## 4. Failure modes: the scope-creep dynamic

The dominant failure mode for Pattern C is scope creep — an adjacent component begins as a coherent Pattern C composition handling non-coordination work, and over deployment lifecycle gradually acquires coordination involvement. The drift is steady because the boundary between "administrative AI" and "coordination AI" is operationally blurry under feature pressure. The boundary inspectability requirement per component (d) is what makes drift visible; without inspectability, drift happens silently, and the deployment discovers the failure only when downstream commitments visibly fail.

Six failure modes recur in this category.

**Coordination state migrates to the component.** The component begins to hold state in categories the substrate is the source of truth for per A2.43–A2.47 — typically through caching, indexing, or "memory" features added incrementally. Component (b) fails; a second authority over the same questions has been created without architectural recognition.

**The component begins answering coordination questions.** Even when the component does not hold the relevant state, it is invoked to answer queries that should be answered from substrate per A2.43–A2.44. The component becomes a coordination-answering authority through operational use rather than architectural commitment.

**The component begins exercising governance authority.** The component makes decisions about coordination state — approving or rejecting substrate writes, modifying authority structures, gating cell execution — outside the orchestration-rule architecture. Component (c) fails; governance per A1.01 is exercised by an entity the architecture does not authorize, and the inspect, modify, and override rights cannot be exercised over its decisions because they did not pass through the substrate.

**The boundary becomes ambiguous over time.** Through accumulated feature additions, integrations, or vendor updates, humans cannot reliably determine which decisions the substrate carries and which the adjacent component handles. Component (d) fails not through any single change but through the cumulative effect of many small ones, which is precisely why architectural inspectability is the defense.

**An operational interface routes coordination queries to the component.** Chatbots, search interfaces, or unified query layers route queries to the component without scope filtering. The architectural failure is upstream of the component itself: the routing layer has erased the distinction the pattern requires.

**The component holds derived coordination state without being modeled as Pattern B.** The component receives derivatives of coordination state — summaries, indexed excerpts, embedded representations — without those derivatives being modeled as Pattern B derived views. The component holds coordination-derived state that is neither authoritative nor architecturally positioned as a derived view, a position the architecture does not name. The fix is to remove the coordination-derived state or re-position the composition as Pattern B per A2.93.

The common thread is that an adjacent component has acquired a position the architecture does not name, and the commitments degrade where the position is unnamed. The architectural defense is the pair of properties named in components (a) and (d): operationally restricted scope plus inspectable boundary.

## 5. Implications and load-bearing connections

Three architectural connections follow from the standalone Pattern C specification.

**Pattern C completes the hybrid-systems-composition decomposition.** A1.16 commits to three positions; without a standalone Pattern C, separate-concern compositions would be ambiguously positioned and the architecture would be incomplete. Pattern C and Pattern A are the load-bearing alternatives at the boundary where coordination involvement is in question: components handle non-coordination concerns in Pattern C or enter cell-mediated involvement in Pattern A, with no architecturally recognized position in between.

**Pattern C extends human governance across separate-concern compositions.** The inspectable-boundary commitment per component (d) preserves the human-governed property per A1.01 when components are present that do not interact with substrate. The inspect right per A2.01 is exercised against the boundary itself: humans can determine, at any time per A2.07, which work is in coordination scope and which is not.

**Pattern C interacts with composition requirements minimally but specifically.** Per A1.13 and the composition-requirements decomposition A2.75–A2.80, Pattern C compositions operate with minimal cross-boundary state by definition, so most composition requirements apply trivially. Requirement E per A2.80 (human-selective composition) applies directly: humans choose to include or exclude adjacent components in the deployment. Requirement A per A2.76 (per-substrate human governance preservation) applies indirectly: if the component crosses into coordination scope and must be re-evaluated as Pattern A, the full composition-requirements machinery applies at that point.

## 6. Operational test

A separate-concern composition instantiates Pattern C if and only if all of the following are true at all times during the deployment's existence:

1. The adjacent component handles work that is operationally distinct from coordination — it does not answer coordination questions, produce coordination decisions, or modify coordination state.
2. The component does not hold state in any of the categories the substrate is the source of truth for per A2.42–A2.48.
3. The component does not exercise authority over substrate content or orchestration rules; coordination decisions flow through cells under orchestration rules per A1.04, and the adjacent component does not bypass this.
4. The boundary between the component's domain and the substrate's domain is inspectable — a human exercising the inspect right per A2.01 can determine, at any time per A2.07 and without scheduling or runtime intermediation, which work the substrate carries and which the adjacent component handles.
5. The four properties above hold as architectural properties of the deployment's design, not as procedural promises; no vendor policy, runtime middleware layer, or operational interface can in principle prevent (1)–(4).
6. Where the pattern is violated, the affected composition is re-evaluated under Pattern A per A2.92 rather than left in Pattern C with ambiguous coordination involvement.

A composition that fails any of (1)–(6) may be a useful configuration, and may be governed in some other sense, but is not Pattern C in the architectural sense. Components that drift across the boundary do not become a fourth pattern; they become Pattern A compositions whose requirements must now be satisfied, or anti-pattern compositions that A2.95 will treat.

## 7. Conclusion

Pattern C in the CKS architecture names the position in which an adjacent AI component coexists with a substrate as a separate concern — handling non-coordination work, holding no coordination state, exercising no governance authority, bounded by an inspectable separation between its domain and the substrate's domain. The pattern is what makes hybrid deployments containing reference-document retrieval, fine-tuned models for non-coordination tasks, semantic search over reference material, and domain-specialized inference systems CKS-coherent without forcing every adjacent capability into Pattern A or Pattern B.

The one-sentence test captures the position operationally: *if an adjacent component handles work operationally distinct from coordination, holds no coordination state, exercises no governance authority over substrate, and the boundary between its domain and the substrate's domain is inspectable by humans exercising the inspect right at any time, the component is in Pattern C; if the component begins answering coordination questions, holding coordination state, exercising authority, or operating with an ambiguous boundary, it has crossed into a position the architecture either does not name or names as Pattern A.*

Naming Pattern C as a standalone commitment is what makes silent scope creep architecturally identifiable rather than only diagnosable in retrospect. The four operational components and the inspectable-boundary requirement together convert "the adjacent component is just for document search" from a deployment claim into an architectural property that can be inspected, exercised, and contested. Subsequent work that adopts the CKS pattern, composes it with adjacent AI capabilities, or argues against the resulting commitments should use "Pattern C" in the sense formalized here. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

---

## Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

## How to cite this note

Li, W. (2026). *Coexistence, Not Coupling: A Standalone Specification of Pattern C (Adjacent Component as Separate Concern) in the Coordination Knowledge Substrate Pattern*. May 5, 2026. ORCID: 0009-0004-8065-3235.